

Práctica completa Django: CRUD, autenticación y recuperación de contraseña

1. Objetivo del proyecto

Crear una aplicación web con Django que permita:

- Trabajar con dos modelos relacionados: **Categoría** y **Producto**.
 - Registrar usuarios.
 - Iniciar sesión.
 - Cerrar sesión.
 - Proteger determinadas vistas para usuarios autenticados.
 - Crear, consultar, editar y eliminar productos y categorías.
 - Recuperar una contraseña olvidada mediante email.
 - Enviar un enlace seguro con token.
 - Introducir una nueva contraseña.
 - Repetir la contraseña para comprobar que coincide.
 - Actualizar correctamente la contraseña del usuario.
-

2. Crear el proyecto

Crear un entorno virtual:

```
python -m venv venv
```

Activarlo en Windows:

```
venv\Scripts\activate
```

En Linux o macOS:

```
source venv/bin/activate
```

Instalar Django:

```
pip install django
```

Crear el proyecto:

django-admin startproject tienda .

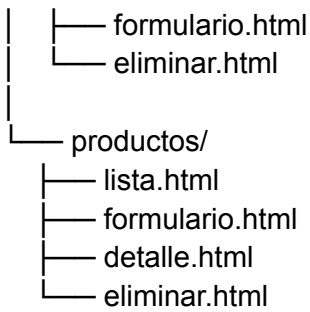
Crear la aplicación:

python manage.py startapp productos

3. Estructura del proyecto

La estructura aproximada será:

```
tienda/
├── manage.py
├── tienda/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── productos/
│   ├── migrations/
│   ├── admin.py
│   ├── apps.py
│   ├── forms.py
│   ├── models.py
│   ├── urls.py
│   └── views.py
├── templates/
│   ├── base.html
│   ├── home.html
│   ├── registration/
│   │   ├── login.html
│   │   ├── register.html
│   │   ├── password_reset_form.html
│   │   ├── password_reset_done.html
│   │   ├── password_reset_email.html
│   │   ├── password_reset_subject.txt
│   │   ├── password_reset_confirm.html
│   │   └── password_reset_complete.html
│   └── categorias/
│       └── lista.html
```



4. Configurar `settings.py`

Abrir:

tienda/settings.py

Añadir nuestra aplicación:

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
  
    "productos",  
]
```

Configurar la carpeta de templates:

```
from pathlib import Path
```

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

En la configuración `TEMPLATES`:

```
TEMPLATES = [  
    {  
        "BACKEND": "django.template.backends.django.DjangoTemplates",  
        "DIRS": [BASE_DIR / "templates"],  
        "APP_DIRS": True,  
        "OPTIONS": {  
            "context_processors": [  
                "django.template.context_processors.request",  
                "django.contrib.auth.context_processors.auth",  
                "django.contrib.messages.context_processors.messages",
```

```
    ],  
    },  
  },  
]
```

Añadir:

```
LOGIN_URL = "login"  
LOGIN_REDIRECT_URL = "home"  
LOGOUT_REDIRECT_URL = "login"
```

5. Configurar el email

Durante el desarrollo utilizaremos el backend de consola.

Añadir al final de `settings.py`:

```
EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend"  
  
DEFAULT_FROM_EMAIL = "no-reply@tienda.com"
```

Con esta configuración el email no se enviará realmente.

El contenido aparecerá en la terminal donde se está ejecutando Django.

6. Crear los modelos

Archivo:

productos/models.py

Código:

```
from django.db import models  
  
class Categoria(models.Model):  
    nombre = models.CharField(max_length=100)  
    descripcion = models.TextField(blank=True)  
    fecha_creacion = models.DateTimeField(auto_now_add=True)  
  
    def __str__(self):  
        return self.nombre
```

```
class Producto(models.Model):
    nombre = models.CharField(max_length=150)
    descripcion = models.TextField(blank=True)
    precio = models.DecimalField(
        max_digits=10,
        decimal_places=2
    )

    categoria = models.ForeignKey(
        Categoria,
        on_delete=models.CASCADE,
        related_name="productos"
    )

    fecha_creacion = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.nombre
```

La relación será:

Categoría 1 ----- N Producto

Una categoría puede tener muchos productos.

Un producto solamente pertenece a una categoría.

7. Crear las migraciones

Ejecutar:

```
python manage.py makemigrations
```

Después:

```
python manage.py migrate
```

8. Registrar los modelos en el administrador

Archivo:

productos/admin.py

Código:

```
from django.contrib import admin
from .models import Categoria, Producto
```

```
admin.site.register(Categoria)
admin.site.register(Producto)
```

Crear administrador:

```
python manage.py createsuperuser
```

9. Crear **forms.py**

Crear:

productos/forms.py

Código:

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
```

```
from .models import Categoria, Producto
```

```
class RegistroForm(UserCreationForm):
    email = forms.EmailField(required=True)
```

```
    class Meta:
        model = User
        fields = [
            "username",
            "email",
            "password1",
            "password2",
        ]
```

```
    def clean_email(self):
        email = self.cleaned_data.get("email")
```

```
if User.objects.filter(email=email).exists():
    raise forms.ValidationError(
        "Ya existe un usuario con este correo electrónico."
    )
```

```
return email
```

```
class CategoriaForm(forms.ModelForm):
```

```
    class Meta:
        model = Categoria
        fields = [
            "nombre",
            "descripcion",
        ]
```

```
class ProductoForm(forms.ModelForm):
```

```
    class Meta:
        model = Producto
        fields = [
            "nombre",
            "descripcion",
            "precio",
            "categoria",
        ]
```

Django `UserCreationForm` ya realiza la comprobación de:

- contraseña;
- repetición de contraseña;
- reglas de seguridad;
- validadores configurados en Django.

10. Crear las vistas

Archivo:

productos/views.py

Código:

```
from django.contrib.auth.decorators import login_required
from django.contrib.auth import login
```

```
from django.shortcuts import (
    render,
    redirect,
    get_object_or_404,
)
```

```
from .models import Categoria, Producto
```

```
from .forms import (
    RegistroForm,
    CategoriaForm,
    ProductoForm,
)
```

11. Página principal

Añadir:

```
def home(request):
    return render(request, "home.html")
```

12. Registro de usuarios

Añadir:

```
def register(request):

    if request.user.is_authenticated:
        return redirect("home")

    if request.method == "POST":

        form = RegistroForm(request.POST)

        if form.is_valid():

            usuario = form.save()

            login(request, usuario)
```

```
return redirect("home")
```

else:

```
form = RegistroForm()
```

```
return render(
    request,
    "registration/register.html",
    {
        "form": form
    }
)
```

Cuando el registro sea correcto:

Usuario registrado



Login automático



Página principal

13. CRUD de categorías

Listar categorías

@login_required

def categoria_lista(request):

```
    categorias = Categoria.objects.all()
```

```
    return render(
        request,
        "categorias/lista.html",
        {
            "categorias": categorias
        }
    )
```

Crear categoría

@login_required

def categoria_crear(request):

```
if request.method == "POST":

    form = CategoriaForm(request.POST)

    if form.is_valid():

        form.save()

        return redirect("categoria_lista")

else:

    form = CategoriaForm()

return render(
    request,
    "categorias/formulario.html",
    {
        "form": form,
        "titulo": "Nueva categoría",
    }
)
```

Editar categoría

```
@login_required
def categoria_editar(request, pk):

    categoria = get_object_or_404(
        Categoria,
        pk=pk
    )

    if request.method == "POST":

        form = CategoriaForm(
            request.POST,
            instance=categoria
        )

        if form.is_valid():

            form.save()

            return redirect("categoria_lista")

    else:
```

```
form = CategoriaForm(
    instance=categoria
)

return render(
    request,
    "categorias/formulario.html",
    {
        "form": form,
        "titulo": "Editar categoría",
    }
)
```

Eliminar categoría

```
@login_required
def categoria_eliminar(request, pk):

    categoria = get_object_or_404(
        Categoria,
        pk=pk
    )

    if request.method == "POST":

        categoria.delete()

        return redirect("categoria_lista")

    return render(
        request,
        "categorias/eliminar.html",
        {
            "categoria": categoria
        }
    )
```

14. CRUD de productos

Listar productos

```
def producto_lista(request):
```

```
productos = Producto.objects.select_related(
    "categoria"
)

return render(
    request,
    "productos/lista.html",
    {
        "productos": productos
    }
)
```

Detalle de un producto

def producto_detalle(request, pk):

```
    producto = get_object_or_404(
        Producto,
        pk=pk
    )
```

```
    return render(
        request,
        "productos/detalle.html",
        {
            "producto": producto
        }
    )
```

Crear producto

@login_required

def producto_crear(request):

```
    if request.method == "POST":
```

```
        form = ProductoForm(request.POST)
```

```
        if form.is_valid():
```

```
            form.save()
```

```
            return redirect("producto_lista")
```

```
    else:
```

```
        form = ProductoForm()

    return render(
        request,
        "productos/formulario.html",
        {
            "form": form,
            "titulo": "Nuevo producto",
        }
    )
```

Editar producto

```
@login_required
def producto_editar(request, pk):

    producto = get_object_or_404(
        Producto,
        pk=pk
    )

    if request.method == "POST":

        form = ProductoForm(
            request.POST,
            instance=producto
        )

        if form.is_valid():

            form.save()

            return redirect("producto_lista")

    else:

        form = ProductoForm(
            instance=producto
        )

    return render(
        request,
        "productos/formulario.html",
        {
            "form": form,
            "titulo": "Editar producto",
```

```
}  
)
```

Eliminar producto

```
@login_required  
def producto_eliminar(request, pk):  
  
    producto = get_object_or_404(  
        Producto,  
        pk=pk  
    )  
  
    if request.method == "POST":  
  
        producto.delete()  
  
        return redirect("producto_lista")  
  
    return render(  
        request,  
        "productos/eliminar.html",  
        {  
            "producto": producto  
        }  
    )
```

15. Archivo completo **urls.py** de la aplicación

Crear:

productos/urls.py

Código:

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
```

```
path(
    "",
    views.home,
    name="home"
),
```

```
path(
    "register/",
    views.register,
    name="register"
),
```

Categorías

```
path(
    "categorias/",
    views.categoria_lista,
    name="categoria_lista"
),
```

```
path(
    "categorias/nueva/",
    views.categoria_crear,
    name="categoria_crear"
),
```

```
path(
    "categorias/<int:pk>/editar/",
    views.categoria_editar,
    name="categoria_editar"
),
```

```
path(
    "categorias/<int:pk>/eliminar/",
    views.categoria_eliminar,
    name="categoria_eliminar"
),
```

Productos

```
path(
    "productos/",
    views.producto_lista,
    name="producto_lista"
),
```

```
path(
    "productos/<int:pk>/",
```

```

        views.producto_detalle,
        name="producto_detalle"
    ),

    path(
        "productos/nuevo/",
        views.producto_crear,
        name="producto_crear"
    ),

    path(
        "productos/<int:pk>/editar/",
        views.producto_editar,
        name="producto_editar"
    ),

    path(
        "productos/<int:pk>/eliminar/",
        views.producto_eliminar,
        name="producto_eliminar"
    ),
]

```

16. URLs principales del proyecto

Abrir:

tienda/urls.py

Código:

```
from django.contrib import admin
```

```

from django.urls import (
    path,
    include,
)

```

```
urlpatterns = [
```

```

    path(
        "admin/",
        admin.site.urls
    ),

```

```
path(
    "",
    include("productos.urls")
),

path(
    "accounts/",
    include("django.contrib.auth.urls")
),

]
```

La línea:

```
path(
    "accounts/",
    include("django.contrib.auth.urls")
)
```

nos proporciona automáticamente las rutas de autenticación de Django.

Entre ellas:

/accounts/login/

/accounts/logout/

/accounts/password_reset/

/accounts/password_reset/done/

/accounts/reset/<uidb64>/<token>/

/accounts/reset/done/

17. Template base

Crear:

templates/base.html

Código:

```
<!DOCTYPE html>
```

```
<html lang="es">
```

<head>

<meta charset="UTF-8">

<title>

{% block title %}

Tienda Django

{% endblock %}

</title>

</head>

<body>

<nav>

Inicio

|

Productos

{% if user.is_authenticated %}

|

Categorías

|

Usuario:

{{ user.username }}

|

<form

method="post"

action="{% url 'logout' %}"

style="display:inline;"

>

```
        {% csrf_token %}

        <button type="submit">
            Logout
        </button>

    </form>

{% else %}

    |

    <a href="{% url 'login' %}">
        Login
    </a>

    |

    <a href="{% url 'register' %}">
        Registrarse
    </a>

{% endif %}

</nav>

<hr>

{% block content %}

{% endblock %}

</body>

</html>
```

18. Página principal

Crear:

templates/home.html

Código:

```
{% extends "base.html" %}
```

```
{% block title %}
```

```
Inicio
```

```
{% endblock %}
```

```
{% block content %}
```

```
<h1>
```

```
    Tienda Django
```

```
</h1>
```

```
{% if user.is_authenticated %}
```

```
    <p>
```

```
        Bienvenido,
```

```
        {{ user.username }}.
```

```
    </p>
```

```
{% else %}
```

```
    <p>
```

```
        Inicia sesión o crea una cuenta.
```

```
    </p>
```

```
{% endif %}
```

```
{% endblock %}
```

19. Template de registro

Crear:

templates/registration/register.html

Código:

```
{% extends "base.html" %}
```

```
{% block title %}
```

```
Registro
```

```
{% endblock %}
```

```
{% block content %}
```

```
<h1>
```

```
    Crear cuenta
```

```
</h1>
```

```
<form method="post">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">
        Registrarse
    </button>

</form>

<p>

    ¿Ya tienes cuenta?

    <a href="{% url 'login' %}">
        Iniciar sesión
    </a>

</p>

{% endblock %}
```

20. Template de login

Crear:

templates/registration/login.html

Código:

```
{% extends "base.html" %}

{% block title %}
Login
{% endblock %}

{% block content %}

<h1>
    Iniciar sesión
</h1>

{% if form.errors %}
```

```
<p>
    Usuario o contraseña incorrectos.
</p>

{% endif %}

<form method="post">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">
        Entrar
    </button>

</form>

<p>

    <a href="{% url 'password_reset' %}">
        ¿Has olvidado tu contraseña?
    </a>

</p>

<p>

    ¿No tienes cuenta?

    <a href="{% url 'register' %}">
        Registrarse
    </a>

</p>

{% endblock %}
```

21. Listar productos

Crear:

templates/productos/lista.html

Código:

```
{% extends "base.html" %}

{% block content %}

<h1>
    Productos
</h1>

{% if user.is_authenticated %}

<a href="{% url 'producto_crear' %}">
    Nuevo producto
</a>

{% endif %}

<hr>

{% for producto in productos %}

    <h2>

        <a href="{% url 'producto_detalle' producto.pk %}">

            {{ producto.nombre }}

        </a>

    </h2>

    <p>
        Categoría:
        {{ producto.categoria.nombre }}
    </p>

    <p>
        Precio:
        {{ producto.precio }} €
    </p>

{% empty %}

    <p>
        No existen productos.
    </p>
```

```
{% endfor %}
```

```
{% endblock %}
```

22. Detalle de producto

Crear:

templates/productos/detalle.html

Código:

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<h1>
```

```
    {{ producto.nombre }}
```

```
</h1>
```

```
<p>
```

```
    <strong>Categoría:</strong>
```

```
    {{ producto.categoria.nombre }}
```

```
</p>
```

```
<p>
```

```
    <strong>Descripción:</strong>
```

```
    {{ producto.descripcion }}
```

```
</p>
```

```
<p>
```

```
    <strong>Precio:</strong>
```

```
    {{ producto.precio }} €
```

```
</p>
```

```
{% if user.is_authenticated %}
```

```
<a href="{% url 'producto_editar' producto.pk %}">
    Editar
</a>

|

<a href="{% url 'producto_eliminar' producto.pk %}">
    Eliminar
</a>

{% endif %}

{% endblock %}
```

23. Formulario de productos

Crear:

templates/productos/formulario.html

Código:

```
{% extends "base.html" %}

{% block content %}

<h1>
    {{ titulo }}
</h1>

<form method="post">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">
        Guardar
    </button>

</form>

<a href="{% url 'producto_lista' %}">
    Cancelar
</a>
```

```
{% endblock %}
```

24. Eliminar producto

Crear:

templates/productos/eliminar.html

Código:

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<h1>
```

```
    Eliminar producto
```

```
</h1>
```

```
<p>
```

```
    ¿Seguro que quieres eliminar
```

```
    <strong>
```

```
        {{ producto.nombre }}
```

```
    </strong>
```

```
    ?
```

```
</p>
```

```
<form method="post">
```

```
    {% csrf_token %}
```

```
    <button type="submit">
```

```
        Sí, eliminar
```

```
    </button>
```

```
</form>
```

```
<a href="{% url 'producto_lista' %}">
```

```
    Cancelar
```

```
</a>
```

```
{% endblock %}
```

25. Lista de categorías

Crear:

templates/categorias/lista.html

Código:

```
{% extends "base.html" %}

{% block content %}

<h1>
    Categorías
</h1>

<a href="{% url 'categoria_crear' %}">
    Nueva categoría
</a>

<hr>

{% for categoria in categorias %}

    <h2>
        {{ categoria.nombre }}
    </h2>

    <p>
        {{ categoria.descripcion }}
    </p>

    <a href="{% url 'categoria_editar' categoria.pk %}">
        Editar
    </a>

    |

    <a href="{% url 'categoria_eliminar' categoria.pk %}">
        Eliminar
    </a>

{% empty %}

<p>
    No existen categorías.
</p>
```

```
{% endfor %}
```

```
{% endblock %}
```

26. Formulario de categorías

Crear:

templates/categorias/formulario.html

Código:

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<h1>
```

```
    {{ titulo }}
```

```
</h1>
```

```
<form method="post">
```

```
    {% csrf_token %}
```

```
    {{ form.as_p }}
```

```
    <button type="submit">
```

```
        Guardar
```

```
    </button>
```

```
</form>
```

```
{% endblock %}
```

27. Eliminar categoría

Crear:

templates/categorias/eliminar.html

Código:

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<h1>
```

```
    Eliminar categoría
```

```
</h1>
```

```
<p>
```

```
    ¿Seguro que quieres eliminar
```

```
    <strong>
```

```
        {{ categoria.nombre }}
```

```
    </strong>
```

```
    ?
```

```
</p>
```

```
<p>
```

```
    Atención: también se eliminarán sus productos.
```

```
</p>
```

```
<form method="post">
```

```
    {% csrf_token %}
```

```
    <button type="submit">
```

```
        Eliminar
```

```
    </button>
```

```
</form>
```

```
{% endblock %}
```

28. Recuperar contraseña

Ahora vamos a implementar el flujo completo de recuperación de contraseña.

Django ya incorpora las vistas necesarias.

El flujo será:

Login

↓
¿Olvidaste tu contraseña?
↓
Introducir email
↓
Django busca una cuenta asociada
↓
Genera UID + token
↓
Envía enlace por email
↓
Usuario abre el enlace
↓
Nueva contraseña
↓
Repetir contraseña
↓
Validación
↓
Actualización de contraseña
↓
Login

29. Formulario para solicitar el email

Crear:

templates/registration/password_reset_form.html

Código:

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<h1>
```

```
    Recuperar contraseña
```

```
</h1>
```

```
<p>
```

```
    Introduce el correo electrónico asociado a tu cuenta.
```

```
</p>
```

```
<form method="post">
```

```
    {% csrf_token %}
```

```
{{ form.as_p }}

<button type="submit">
    Recuperar contraseña
</button>

</form>

{% endblock %}
```

Django utilizará automáticamente un campo:

email

30. Página después de solicitar el email

Crear:

templates/registration/password_reset_done.html

Código:

```
{% extends "base.html" %}

{% block content %}

<h1>
    Solicitud procesada
</h1>

<p>
    Si existe una cuenta asociada al correo indicado,
    recibirás un mensaje con las instrucciones para cambiar
    tu contraseña.
</p>

<p>

    <a href="{% url 'login' %}">
        Volver al login
    </a>

</p>

{% endblock %}
```

Es preferible no mostrar:

Ese email no existe

de forma pública.

Esto evita revelar qué emails están registrados en nuestra aplicación.

31. Email de recuperación

Crear:

templates/registration/password_reset_email.html

Código:

Hola {{ user.get_username }},

Hemos recibido una solicitud para cambiar la contraseña de tu cuenta.

Pulsa en el siguiente enlace:

{{ protocol }}://{{ domain }}{% url 'password_reset_confirm' uidb64=uid token=token %}

Si no has solicitado este cambio puedes ignorar este mensaje.

32. Asunto del correo

Crear:

templates/registration/password_reset_subject.txt

Contenido:

Recuperación de contraseña

33. Formulario de nueva contraseña

Crear:

templates/registration/password_reset_confirm.html

Código:

```
{% extends "base.html" %}

{% block content %}

<h1>
    Nueva contraseña
</h1>

{% if validlink %}

    <p>
        Introduce tu nueva contraseña dos veces.
    </p>

    <form method="post">

        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">
            Cambiar contraseña
        </button>

    </form>

{% else %}

    <p>
        El enlace de recuperación no es válido o ha caducado.
    </p>

    <a href="{% url 'password_reset' %}">
        Solicitar otro enlace
    </a>

{% endif %}

{% endblock %}
```

Django mostrará automáticamente dos campos:

Nueva contraseña

Repetir nueva contraseña

Internamente corresponden a:

new_password1
new_password2

Django comprobará automáticamente que coincidan.

34. Confirmación del cambio

Crear:

templates/registration/password_reset_complete.html

Código:

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<h1>
```

```
    Contraseña actualizada
```

```
</h1>
```

```
<p>
```

```
    Tu contraseña se ha cambiado correctamente.
```

```
</p>
```

```
<a href="{% url 'login' %}">
```

```
    Iniciar sesión
```

```
</a>
```

```
{% endblock %}
```

35. ¿Cómo funciona internamente el reset?

Cuando se solicita recuperar una contraseña, Django trabaja con:

User

↓

uidb64

↓

token

El enlace puede tener esta apariencia:

`http://127.0.0.1:8000/accounts/reset/MQ/cx1abc-xxxxxxx/`

El `uidb64` identifica al usuario de manera codificada.

El `token` permite validar la solicitud.

Django comprueba automáticamente:

- que el usuario exista;
 - que el token sea válido;
 - que el token corresponda al usuario;
 - que el enlace siga siendo válido;
 - que las dos contraseñas coincidan;
 - que la nueva contraseña cumpla los validadores.
-

36. No actualizar contraseñas incorrectamente

Nunca debemos hacer:

```
usuario.password = nueva_password
usuario.save()
```

Eso almacenaría incorrectamente el valor.

Si alguna vez cambiamos una contraseña manualmente debemos usar:

```
usuario.set_password(nueva_password)
usuario.save()
```

Django utiliza hashing para almacenar la contraseña.

Por ejemplo, en la base de datos puede aparecer algo parecido a:

`pbkdf2_sha256$...`

Nunca aparecerá:

`123456`

37. Ejecutar el servidor

Ejecutar:

```
python manage.py runserver
```

Abrir:

```
http://127.0.0.1:8000/
```

38. Probar el registro

Entrar en:

```
http://127.0.0.1:8000/register/
```

Crear un usuario.

Ejemplo:

Usuario:

```
juan
```

Email:

```
juan@email.com
```

Contraseña:

```
UnaClaveSegura2026!
```

Repetir contraseña:

```
UnaClaveSegura2026!
```

39. Probar login

Ir a:

```
http://127.0.0.1:8000/accounts/login/
```

Introducir:

```
juan
```

```
UnaClaveSegura2026!
```

El usuario debería acceder correctamente.

40. Probar logout

En el menú pulsar:

Logout

La sesión deberá cerrarse.

41. Probar una vista protegida

Intentar acceder sin haber iniciado sesión a:

`http://127.0.0.1:8000/productos/nuevo/`

Django deberá redirigir automáticamente a algo parecido a:

`/accounts/login/?next=/productos/nuevo/`

Después del login regresará a la página solicitada.

42. Probar recuperación de contraseña

Desde:

`/accounts/login/`

pulsar:

¿Has olvidado tu contraseña?

Se abrirá:

`/accounts/password_reset/`

Introducir:

`juan@email.com`

Pulsar:

Recuperar contraseña

43. Ver el email

Como estamos usando:

```
EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend"
```

el email aparecerá en la terminal.

Puede ser similar a:

Content-Type: text/plain

Recuperación de contraseña

Hola juan,

Hemos recibido una solicitud para cambiar
la contraseña de tu cuenta.

Pulsa en el siguiente enlace:

<http://127.0.0.1:8000/accounts/reset/MQ/xxxxx-xxxxx/>

Copiar el enlace.

44. Introducir nueva contraseña

Abrir el enlace.

Django mostrará:

Nueva contraseña:

Repetir nueva contraseña:

Por ejemplo:

NuevaClave2026!

NuevaClave2026!

Si ambas coinciden y cumplen los requisitos:

Contraseña actualizada

45. Comprobar la contraseña

Volver a:

/accounts/login/

Intentar entrar con la contraseña antigua.

No debería funcionar.

Después utilizar:

NuevaClave2026!

El login deberá funcionar correctamente.

46. Configurar email real mediante SMTP

Una vez que todo funciona con la consola se puede configurar un servidor SMTP.

Por ejemplo, conceptualmente:

```
EMAIL_BACKEND = "django.core.mail.backends.smtp.EmailBackend"
```

```
EMAIL_HOST = "smtp.servidor.com"
```

```
EMAIL_PORT = 587
```

```
EMAIL_USE_TLS = True
```

```
EMAIL_HOST_USER = "correo@dominio.com"
```

```
EMAIL_HOST_PASSWORD = "contraseña-aplicacion"
```

```
DEFAULT_FROM_EMAIL = EMAIL_HOST_USER
```

No es recomendable escribir credenciales directamente en `settings.py`.

Es mejor utilizar variables de entorno.

Por ejemplo:

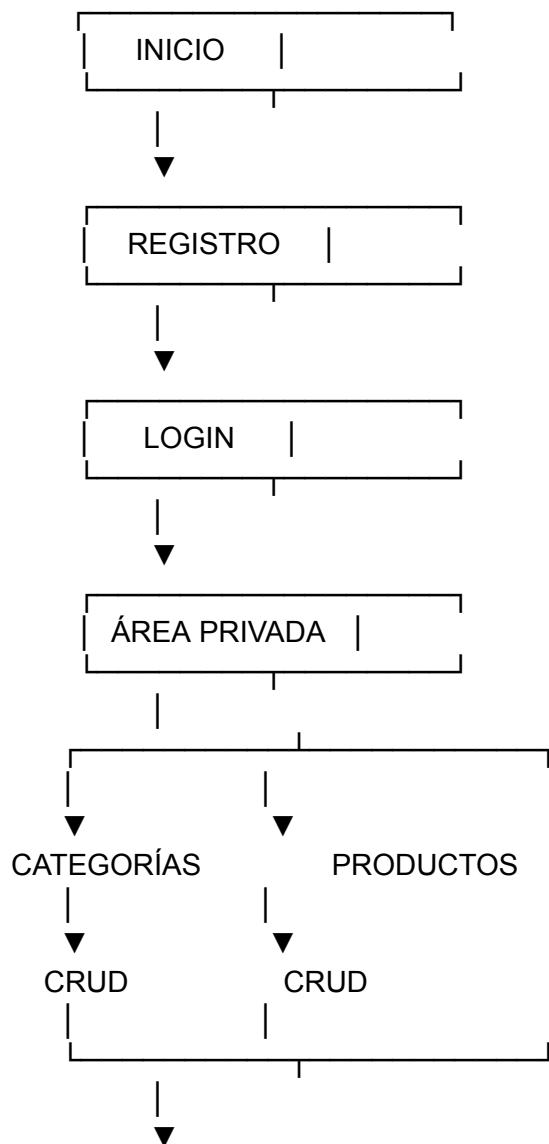
```
import os
```

```
EMAIL_HOST_USER = os.getenv("EMAIL_HOST_USER")
```

```
EMAIL_HOST_PASSWORD = os.getenv(  
    "EMAIL_HOST_PASSWORD"  
)
```

47. Flujo completo del sistema

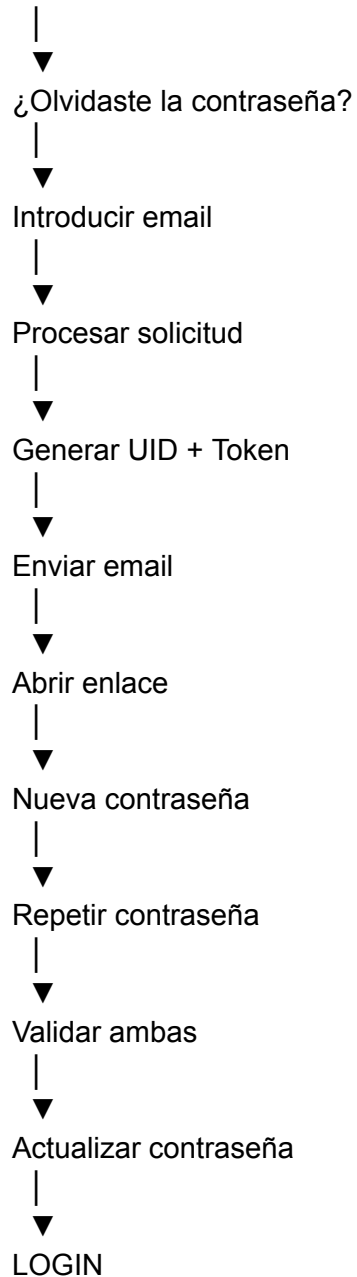
El proyecto debe permitir:



LOGOUT

Y para recuperar la contraseña:

LOGIN



48. Aspectos importantes para evaluar

Se puede evaluar al alumno comprobando los siguientes puntos:

Modelos

- Existen al menos dos modelos.
- Existe una relación `ForeignKey`.
- Las migraciones funcionan correctamente.

CRUD

- Crear.
- Consultar.
- Editar.
- Eliminar.
- Listar.

Autenticación

- Registro.
- Login.
- Logout.
- Uso de `request.user`.
- Protección mediante `login_required`.

Recuperación de contraseña

- Formulario para solicitar el email.
- Generación de token.
- Envío del enlace.
- Comprobación del token.
- Introducción de nueva contraseña.
- Repetición de contraseña.
- Actualización del password.
- Login posterior con la contraseña nueva.

Seguridad

- Uso de `{% csrf_token %}`.
- Contraseñas almacenadas mediante Django.
- No almacenar contraseñas en texto plano.
- Uso del sistema oficial de autenticación.
- Protección de vistas privadas.

49. Posibles ampliaciones

Una vez terminada la versión básica se pueden añadir nuevas funcionalidades:

- Bootstrap.
 - Imágenes para productos.
 - Buscador.
 - Filtros por categoría.
 - Paginación.
 - Perfil de usuario.
 - Editar perfil.
 - Cambiar contraseña estando autenticado.
 - Roles de usuario.
 - Permisos.
 - Productos propiedad de un usuario.
 - API REST.
 - Django REST Framework.
 - PostgreSQL.
 - Variables de entorno.
 - Docker.
 - Despliegue en producción.
-

50. Resultado final esperado

Al terminar, un usuario deberá poder realizar este proceso completo:

Registrarse



Iniciar sesión



Crear categorías



Crear productos asociados a categorías



Editar productos



Eliminar productos



Cerrar sesión



Olvidar la contraseña



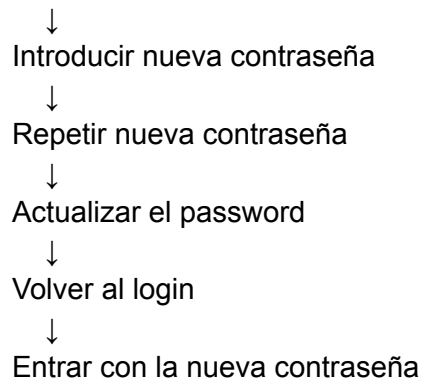
Solicitar recuperación mediante email



Recibir un enlace con token



Acceder al formulario de cambio



Con esto se dispone de una aplicación Django funcional con relaciones entre modelos, CRUD, autenticación completa y recuperación segura de contraseña.